

Universidad Tecnológica Nacional
Tecnicatura Universitaria en Programación
Programación 2

FoodStoreTPI

Sistema de Gestión de Pedidos de Comida

Integrantes

Gimenez Cristhian Santiago
Luis Castro
Enzo Vazquez

Trabajo Práctico Integrador
Documentación académica y técnica con devolución evaluativa

25 de junio de 2026

Índice

Resumen	3
Introducción	4
Objetivos	5
Marco Teórico	6
Descripción Del Sistema	7
Arquitectura Y Modelo	8
Reglas De Negocio	9
Trazabilidad Con La Consigna	10
Devolución Evaluativa	11
Dificultades Y Soluciones	12
Conclusión	13
Referencias	14

Resumen

FoodStoreTPI es una aplicación de consola desarrollada en Java para gestionar categorías, productos, usuarios y pedidos de un negocio de comidas. El proyecto responde al Trabajo Práctico Integrador de Programación 2 de la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional. Su diseño se apoya en Programación Orientada a Objetos, separación por capas, colecciones en memoria, enumeraciones, una interfaz común para el cálculo de totales y excepciones propias para controlar errores de dominio.

La presente documentación integra la consigna académica con la descripción técnica del README del proyecto. Además, incorpora una devolución evaluativa con fortalezas, aspectos a completar antes de la entrega y recomendaciones de mejora, siguiendo criterios de presentación y citación compatibles con normas APA.

Palabras clave: Java, Programación Orientada a Objetos, consola, colecciones, excepciones, pedidos.

Introducción

El sistema FoodStoreTPI tiene como propósito resolver, desde una aplicación de consola, la gestión básica de un negocio de comidas. La consigna solicita operaciones CRUD completas sobre categorías, productos, usuarios y pedidos, junto con validaciones de entrada, manejo de excepciones propias y una organización clara entre modelo, servicios y vista por consola.

El proyecto documentado implementa una estructura de paquetes con entidades, enumeraciones, excepciones, interfaces, servicios y vista. Esta organización permite distinguir las responsabilidades centrales: las entidades representan el dominio, los servicios coordinan las reglas de negocio y la clase principal gestiona la interacción con el usuario mediante menus.

Objetivos

Objetivo general: documentar y evaluar el sistema FoodStoreTPI según la consigna del Trabajo Práctico Integrador de Programación 2, utilizando un formato académico con referencias en estilo APA.

Objetivos específicos: describir el alcance funcional del sistema; explicar las tecnologías y conceptos aplicados; detallar la arquitectura y el modelo de datos; vincular las funcionalidades con las historias de usuario de la consigna; identificar fortalezas, riesgos y mejoras necesarias para una entrega mas solida.

Marco Teórico

Java es el lenguaje utilizado para construir la aplicación. En este proyecto se aplican conceptos centrales de Programación Orientada a Objetos: encapsulamiento mediante atributos y métodos, herencia a partir de una clase base comun, asociaciones entre entidades, enumeraciones para estados controlados y polimorfismo mediante la interfaz Calculable. La documentación oficial de Java SE 21 define el marco de referencia del lenguaje y de las bibliotecas utilizadas, como `java.util` para colecciones y `java.time` para fechas.

Las colecciones permiten administrar objetos en memoria durante la ejecución del programa. En FoodStoreTPI se usan listas para mantener categorías, productos, usuarios y pedidos activos o eliminados lógicamente. Esta decision coincide con el alcance de la consigna, que no requiere persistencia en base de datos y prioriza el aprendizaje de estructuras dinámicas.

El manejo de excepciones propias permite convertir errores del dominio en mensajes controlados para el usuario. Ejemplos relevantes son `DatosInvalidosException`, `EntidadDuplicadaException`, `EntidadNoEncontradaException` y `StockInvalidoException`. Esta estrategia evita cierres inesperados y mejora la robustez del flujo de menu.

Apache Ant y NetBeans aparecen como herramientas de construcción y entorno de desarrollo. Ant automatiza tareas de compilación mediante un archivo `build.xml`, mientras que NetBeans permite trabajar con proyectos Java de escritorio y ejecutar el ciclo de desarrollo desde el IDE.

Descripción Del Sistema

FoodStoreTPI gestiona cuatro núcleos funcionales: categorías, productos, usuarios y pedidos. Las categorías organizan el catalogo; los productos incluyen precio, stock, imagen, disponibilidad y categoría asociada; los usuarios permiten vincular clientes con pedidos; los pedidos agrupan detalles, forma de pago, estado, fecha y total.

El menu principal ofrece acceso a gestión de categorías, productos, usuarios y pedidos. Cada módulo contiene opciones para crear, listar, editar y eliminar lógicamente registros. El sistema trabaja con datos en memoria volatil, por lo que la información se pierde al cerrar la aplicación. Esta limitacion es aceptable dentro del alcance de Programación 2, aunque debe explicitarse en la entrega.

Arquitectura Y Modelo

La arquitectura se organiza por paquetes: entities para el modelo de dominio, enums para valores cerrados, excepciones para errores propios, interfaces para contratos de comportamiento, services para lógica de negocio y vista para la interacción por consola. Esta separación coincide con la arquitectura recomendada por la consigna: modelo, servicios/lógica y UI.

La clase abstracta Base concentra atributos comunes: id, eliminado y createdAt. Categoría, Producto, Usuario, Pedido y DetallePedido heredan de Base, lo que evita duplicación y refuerza el uso de herencia. Pedido implementa Calculable, interfaz que define el método calcularTotal, utilizado para establecer el total según los detalles activos.

Los estados del pedido se modelan mediante el enum Estado con los valores PENDIENTE, CONFIRMADO, TERMINADO y CANCELADO. La forma de pago se modela con FormaPago: EFECTIVO, TARJETA y TRANSFERENCIA. Los roles se modelan con Rol: ADMIN y USUARIO. Esta elección evita cadenas libres y reduce errores por valores inválidos.

Reglas De Negocio

El sistema define reglas consistentes con la consigna. Las categorías no pueden tener nombre vacío ni duplicado entre registros activos. Los productos requieren nombre, precio mayor a cero, stock no negativo y categoría activa. Los usuarios requieren datos obligatorios y correo electrónico único. Los pedidos no pueden crearse sin usuario ni sin productos, validan stock suficiente y descuentan stock al registrar la operación.

Todas las bajas se implementan como eliminación lógica mediante el atributo eliminado. Esta decisión conserva historial y evita romper referencias existentes, especialmente en productos incluidos en detalles de pedidos o usuarios asociados a compras previas.

Una mejora recomendable es revisar la consistencia entre la consigna y la implementación del método de Pedido. La consigna menciona `addDetallePedido(...)`, mientras que el README documenta `addDetalle(DetallePedido)`. Si el código funciona correctamente, la diferencia de nombre no impide el objetivo pedagógico, pero conviene justificarla o adaptar el nombre para alinearse mejor con el UML indicado.

Trazabilidad Con La Consigna

Requisito de la consigna	Evidencia en el README	Estado
CRUD de categorías	CategoríaService: crear, listar, buscar, editar y eliminar	Cumple
CRUD de productos	ProductoService con validaciones de precio, stock y categoría	Cumple
CRUD de usuarios	UsuarioService con unicidad de email y baja lógica	Cumple
Gestión de pedidos	GestorPedidos registra pedidos, detalles, stock y total	Cumple
Colecciones en memoria	Uso de ArrayList y datos volatiles	Cumple
Excepciones propias	Excepciones para datos invalidos, duplicados, no encontrados y stock	Cumple
Interfaz Calculable	Pedido implementa calcularTotal()	Cumple
Documentación PDF	Este documento cubre la documentación académica y técnica	Cumple
Capturas y video	No fueron provistos en los insumos analizados	Pendiente
Tests unitarios	Carpeta test vacia documentada como pendiente	Pendiente
Java 21	README indica Java 25 compatible con Java 21+	Revisar

Devolución Evaluativa

Fortalezas. El proyecto muestra una comprensión adecuada de Programación Orientada a Objetos: existe una clase base abstracta, entidades diferenciadas, enums para valores cerrados, interfaz para cálculo de totales y servicios que separan la lógica del menu. La organización por paquetes facilita la lectura y coincide con la estructura solicitada por la cátedra. También es positivo que el README documente reglas de negocio, flujo de creación de pedido y alcance del almacenamiento en memoria.

Cumplimiento funcional. Según la información disponible, el sistema cubre las épicas principales de la consigna: categorías, productos, usuarios y pedidos. La creación de pedidos contempla selección de usuario, forma de pago, carga de productos, validación de stock, descuento automático y cálculo del total. Estas decisiones responden bien al backlog mínimo.

Aspectos a corregir antes de entregar. La consigna indica Java 21 como lenguaje/base de ejecución, mientras que el README menciona Java 25 con compatibilidad Java 21+. Para evitar problemas de corrección, conviene compilar y documentar explícitamente el proyecto con JDK 21 o aclarar que source/target están configurados para una versión aceptada por la cátedra. También se debe completar el enlace al video demostrativo y agregar capturas de pantalla del sistema funcionando, ya que la consigna las solicita dentro de la documentación académica.

Riesgos técnicos. El README informa que no hay tests unitarios y detecta un posible bug en Base.java: el constructor Base(long id) debería inicializar createdAt con LocalDateTime.now(). Esta observación es importante porque createdAt forma parte del modelo base y puede afectar listados, depuración o futuras validaciones. Se recomienda corregirlo antes de la entrega final.

Recomendaciones. Agregar pruebas básicas con JUnit 5 para servicios de categorías, productos, usuarios y pedidos; verificar que las bajas lógicas no rompan referencias; validar cantidades mayores a cero al crear detalles; documentar comandos exactos de ejecución; y anexar capturas del menu principal, CRUD de productos, registro de usuario y creación completa de pedido.

Dificultades Y Soluciones

Una dificultad esperable en este tipo de sistema es mantener consistencia entre objetos relacionados sin base de datos. La solución aplicada consiste en trabajar con referencias entre entidades y en usar bajas lógicas para conservar historial. Otra dificultad es evitar estados invalidos durante la creacion de pedidos; el proyecto lo aborda validando stock antes de descontarlo y usando excepciones para cancelar operaciones incorrectas.

Tambien se observa el desafio de separar la lógica de negocio del menu. El uso de servicios especificos para categorías, productos, usuarios y pedidos ayuda a que la vista se concentre en pedir datos y mostrar resultados, mientras que las validaciones quedan centralizadas.

Conclusión

FoodStoreTPI presenta una base técnica sólida para el Trabajo Práctico Integrador de Programación 2. La solución cumple el núcleo funcional requerido, utiliza Programación Orientada a Objetos, colecciones, excepciones propias, interfaces y una organización modular acorde con la consigna.

Para fortalecer la entrega final, se recomienda alinear la versión de Java con lo solicitado, corregir el constructor de Base si el bug esta presente en el código, agregar pruebas unitarias mínimas e incorporar capturas, enlace al repositorio y video demostrativo. Con esos ajustes, el proyecto quedaria mejor preparado tanto para la evaluacion académica como para su mantenimiento futuro.

Referencias

Apache Software Foundation. (2026). Apache Ant manual. <https://ant.apache.org/manual/index.html>

Apache Software Foundation. (2026). Apache NetBeans. <https://netbeans.apache.org/front/main/index.html>

Oracle. (2023). Collection (Java SE 21 & JDK 21). <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Collection.html>

Oracle. (2023). LocalDateTime (Java SE 21 & JDK 21). <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/LocalDateTime.html>

Universidad Tecnológica Nacional. (2026). Trabajo Práctico Integrador - Programación 2: Food Store - Sistema de Gestión de Pedidos de Comida [Consigna de cátedra].

FoodStoreTPI. (2026). README del proyecto: Sistema de Gestión de Pedidos de Comida [Documentación técnica no publicada].